416 Knicks - API Contract

The Knicks

Contents

Summary

For Build 2510.3

Author: Jerry Zhu

Last Updated: 10/18/2025

This document describes the API contracts for our backend to be consumed from the frontend, including the data structures, API endpoints and how to use them.

The API client library is in `frontend_client/src/api/client.ts` and should be used as the final reference.

To update this document, please install Pandoc and/or LaTeX and run the following command:

```
pandoc api-contract.md -f markdown --toc --toc-depth=4 --standalone -o api-contract.pdf
```

API Contracts

Data Structures

These are the data structures representing the JSON responses that the backend will return.

NOTE: This document only defines data structures, we define ourselves.

EAVS Data

These are the data structures relating to EAVS Data for 2016 -> 2024, fitting under the format of the 2024 format.

For earlier years, we have done our best interpretation of remapping the older EAVS data columns to the most recent format, so that all of the years have a consistent format that is easier to compare.

`BallotStatisticsModel`

This data structure holds columns `C8a` and `C3a`, which is used for counting total drop box ballots.

```
BallotStatisticsModel {
                          //                              2 . 3 . 5
    fullRegionId: string; // A full padded-out fipsCode ST-COUNTY-RESERVED
    regionName: string;
    dropboxBallots: int;
    totalBallotsCast: int;
}
```

Related Endpoints N/A

`MailBallotRejectionStatisticsModel`

This data structure holds the columns from `C9a` to `C9q`, which is used for counting mail ballot rejection reasons.

```
MailBallotRejectionStatisticsModel {
                          //                              2 . 3 . 5
    fullRegionId: string; // A full padded-out fipsCode ST-COUNTY-RESERVED
    countyName: string;
    rejectTotal: int;
```

```
    rejectLate: int;
    rejectNoSignature: int;
    rejectNoWitnessSignature: int;
    rejectSignatureMismatch: int;
    rejectUnofficialEnv: int;
    rejectBallotMissing: int;
    rejectNoSecrecyEnvironment: int;
    rejectMultipleInEnvironment: int;
    rejectUnsealedEnvironment: int;
    rejectNoPostMark: int;
    rejectNoAddress: int;
    rejectVoterDeceased: int;
    rejectDuplicateVote: int;
    rejectMissingDocumentation: int;
    rejectNotEligible: int;
    rejectNoApplication: int;
    rejectOther: int;
}
```

Related Endpoints

- GET /state/{fipsCode}/mail-ballot-rejections
- GET /state/{fipsCode}/{countyFipsCode}/mail-ballot-rejections

`PollbookDeletionStatisticsModel`

This data structure holds the columns from `A12a` to `A12h`, which is used for counting poll book deletion reasons.

```
PollbookDeletionStatisticsModel {
                          //                              2 . 3 . 5
    fullRegionId: string; // A full padded-out fipsCode ST-COUNTY-RESERVED
    countyName: string;
    totalRemoved: int;
    removedReasonMoved: int;
    removedReasonDeceased: int;
    removedReasonFelony: int;
    removedReasonFailedToConfirm: int;
    removedReasonIncompetent: int;
    removedReasonRequested: int;
    removedReasonDuplicate: int;
    removedOther: int;
}
```

Related Endpoints

- GET /state/{fipsCode}/pollbook-deletions

- GET /state/{fipsCode}/{countyFipsCode}/pollbook-deletions

`ProvisionalBallotStatisticsModel`

This data structure holds the columns from `E1a` to `E2i`, which is used for provisional ballot qualification reasons.

```
ProvisionalBallotStatisticsModel {
                          //                              2 . 3 . 5
    fullRegionId: string; // A full padded-out fipsCode ST-COUNTY-RESERVED
    countyName: string;
    totalBallotsCast: int;
    ballotReasonNotOnList: int;
    ballotReasonNoIdAvailable: int;
    ballotReasonChallengedByOfficial: int;
    ballotReasonChallengedByOther: int;
    ballotReasonWrongPrecinct: int;
    ballotReasonNotUpdatedAddress: int;
    ballotReasonDidNotSurrender: int;
    ballotReasonExtendedVotingHours: int;
    ballotReasonSameDayRegistration: int;
    ballotReasonOther: int;
}
```

Related Endpoints

- GET /state/{fipsCode}/provisional-ballots
- GET /state/{fipsCode}/{countyFipsCode}/provisional-ballots

`VoterRegistrationStatisticsModel`

This data structure holds the columns from `A1a` to `A1c`, which is used for counting broad voter registration statistics.

```
VoterRegistrationStatisticsModel {
                          //                              2 . 3 . 5
    fullRegionId: string; // A full padded-out fipsCode ST-COUNTY-RESERVED
    countyName: string;
    total: int;
    active: int;
    inactive: int;
}
```

Related Endpoints

- GET /state/{fipsCode}/voter-registration-count

- GET /state/{fipsCode}/{countyFipsCode}/voter-registration-count

`ViewStateYearSummaryModel`

This data structure holds some summary data and calculated percentages for primarily voter turnout details for a given year of a state.

Intended to fulfill usecase GUI-21.

```
ViewStateYearSummaryModel {
    stateFipsCode: int; // This is unpadded.
    stateCode: string; // Postal code
    stateName: string;
    forYear: int;
    activeRegistered: int;
    inactiveRegistered: int;
    totalRegistered: int;
    totalBallotsCast: int;
    earlyVotingTotal: int;
    ballotsByMail: int;
    totalProvisionalBallotsCast: int;
    activeVoterRate: double;
    inactiveVoterRate: double;
    turnOutRate: double;
    earlyVotingShareRate: double;
    mailinBallotVotingShareRate: double;
}
```

Related Endpoints

- GET /state/{fipsCode}/year-summary
- GET /state/{fipsCode}/year-summary/{year}

Geometry

All the data structures here relate to *EAVS* GeoUnit boundary data. Primarily centroids and the boundary polygons.

`GeoUnitCentroidModel`

This is the data structure used to define the center of an *EAVS* GeoUnit, which is typically a county.

It is populated through as part of the preprocessing steps.

```
GeoUnitCentroidModel {
    fullRegionId: string; /// fully 10 digit padded fips code
    countyName:   string;
    centerX:      float;  /// centerX and centerY are longitude and latitude.
    centerY:      float;
}
```

Related Endpoints

- GET /state/{fipsCode}/centroids

`GeoJSON`

We do not define this structure here, so please refer to the official GeoJSON Specification.

Related Endpoints

- GET /state/{fipsCode}/geometry

Voter Equipment

All the data structures here relate to voting equipment usecases.

`VoterEquipmentModel`

This is the data structure used to define a voting equipment machine by make, type, and manufacturer.

It is based on the class-wide shared data spreadsheet, and is synced from it through a Python script.

This structure is currently read from a CSV file, instead of being retrieved from a database.

```
VoterEquipmentModel {
    manufacturer: string;
    equipmentType: string;
    modelName: string;

    // NOTE(jerry):
    // These are intentionally optional. We distinguish the state of
    // not knowing the value vs. a falsy value.

    discontinued: boolean?;
    firstManufactured: int?; // Year
```

```
    lastManufactured: int?; // Year
    operatingSystem: string?;

    vvpat: bool?; // Voter Verified Paper Audit Trail
                  // whether the machine produces a paper receipt
                  // of the users' digital ballot.
                  //
                  // Can be used for recounting.
    certificationLevel: string?;
    securityRiskDescription: string?;
}
```

Related Endpoints

- GET /votingequipment/
- GET /votingequipment/by-manufacturer/{manufacturer}
- GET /votingequipment/by-type/{type}
- GET /votingequipment/{manufacturer}/{model}

Detailed Voter Registration

All the data structures defined here pertain to *Detailed Voter Registration* usecases.

`VoterRegistrationDataModel`

This is the data structure used to define a registered voter's information.

It contains enough to identify the person, and be used as part of a *Display Registered Voters* functionality.

```
VoterRegistrationDataModel {
    regionId: string;
    firstName: string;
    middleName: string;
    lastName: string;
    partyAffiliation: string;
    status: string;
}
```

Related Endpoints

- GET /voter-registration/

Endpoints

These are the endpoints with their appropriate `client.ts` function name for invocation.

EAVS Data

GET `/state/{fipsCode}/provisional-ballots?year&aggregate`

Description:

This endpoint returns the provisional ballot statistics for a specific state and all of its counties, optionally allowing a request for a specific year.

The corresponding client library function is: `getProvisionalBallots`

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02` `{year}` - This is a **query parameter** which is an integer describing the year of query. The *default value* is `2024`. `{aggregate}` - This is a **query parameter** which is a boolean describing whether to aggregate the columns as a sum or not. The *default value* is `false`.

Returns:

A list of ProvisionalBallotStatisticsModel.

If `aggregate` is specified as true, then it will return a list containing a **singular** element representing the aggregate sum of all the data columns for that state.

GET `/state/{fipsCode}/{countyFipsCode}/provisional-ballots?year`

Description:

This endpoint returns the provisional ballot statistics for a specific state and county, optionally allowing a request for a specific year.

The corresponding client library function is: `getProvisionalBallotsByCounty`.

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02`

`{year}` - This is a **query parameter** which is an integer describing the year of query. The *default value* is `2024`.

Returns:

A ProvisionalBallotStatisticsModel.

GET `/state/{fipsCode}/voter-registration-count?year&aggregate`

Description:

This endpoint returns the broad voter registration statistics for a specific state and all of its counties, optionally allowing a request for a specific year.

The corresponding client library function is: `getVoterRegistrationCounts`

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02` `{year}` - This is a **query parameter** which is an integer describing the year of query. The *default value* is `2024`. `{aggregate}` - This is a **query parameter** which is a boolean describing whether to aggregate the columns as a sum or not. The *default value* is `false`.

Returns:

A list of VoterRegistrationStatisticsModel.

If `aggregate` is specified as true, then it will return a list containing a **singular** element representing the aggregate sum of all the data columns for that state.

GET `/state/{fipsCode}/{countyFipsCode}/voter-registration-count?year`

Description:

This endpoint returns the broad voter registration statistics for a specific state and county, optionally allowing a request for a specific year.

The corresponding client library function is: `getVoterRegistrationCountsByCounty`.

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02` `{year}` - This is a **query parameter** which is an integer describing the year of query. The *default value* is `2024`.

Returns:

A VoterRegistrationStatisticsModel.

GET `/state/{fipsCode}/pollbook-deletions?year&aggregate`

Description:

This endpoint returns the poll book deletion statistics for a specific state and all of its counties, optionally allowing a request for a specific year.

The corresponding client library function is: `getPollbookDeletions`

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02` `{year}` - This is a **query parameter** which is an integer describing the year of query. The *default value* is `2024`. `{aggregate}` - This is a **query parameter** which is a boolean describing whether to aggregate the columns as a sum or not. The *default value* is `false`.

Returns:

A list of PollbookDeletionStatisticsModel.

If `aggregate` is specified as true, then it will return a list containing a **singular** element representing the aggregate sum of all the data columns for that state.

GET `/state/{fipsCode}/{countyFipsCode}/pollbook-deletions?year`

Description:

This endpoint returns the pollbook deletion statistics for a specific state and county, optionally allowing a request for a specific year.

The corresponding client library function is: `getPollbookDeletionsByCounty`

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02` `{year}` - This is a **query parameter** which is an integer describing the year of query. The *default value* is `2024`.

Returns:

A PollbookDeletionStatisticsModel.

GET `/state/{fipsCode}/mail-ballot-rejections?year&aggregate`

Description:

This endpoint returns the mail ballot rejection statistics for a specific state and all of its counties, optionally allowing a request for a specific year.

The corresponding client library function is: `getMailBallotRejections`

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02` `{year}` - This is a **query parameter** which is an integer describing the year of query. The *default value* is `2024`. `{aggregate}` - This is a **query parameter** which is a boolean describing whether to aggregate the columns as a sum or not. The *default value* is `false`.

Returns:

A list of MailBallotRejectionStatisticsModel.

If `aggregate` is specified as true, then it will return a list containing a **singular** element representing the aggregate sum of all the data columns for that state.

GET `/state/{fipsCode}/{countyFipsCode}/mail-ballot-rejections?year`

Description:

This endpoint returns the mail ballot rejection statistics for a specific state and county, optionally allowing a request for a specific year.

The corresponding client library function is: `getMailBallotRejectionsByCounty`

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02` `{year}` - This is a **query parameter** which is an integer describing the year of query. The *default value* is `2024`.

Returns:

A MailBallotRejectionStatisticsModel.

GET `/state/{fipsCode}/year-summary`

Description:

This endpoint will return all the voting summary data for a specified state for all the recorded years based on EAVS data.

The corresponding client library function is: `getViewStateYearSummaryByState`

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02`

Returns:

A list of ViewStateYearSummaryModel for the following years:

- 2024
- 2022
- 2020
- 2018
- 2016

GET `/state/{fipsCode}/year-summary/{year}`

Description:

This endpoint will return the voting summary data for a specified state and year based on the availability from the EAVS data sources.

The corresponding client library function is: `getViewStateYearSummaryByStateForYear`

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02`

`{year}` - This is a **path parameter** which is an integer representing the year of query. The year must be one of the valid years of which there is EAVS data, which is one of the following:

- 2024
- 2022
- 2020
- 2018

- 2016

Returns:

A ViewStateYearSummaryModel for a specific year.

Geometry

GET `/state/{fipsCode}/geometry`

Description:

This endpoint will return the boundary data of a specified state, identified by its FIPS code as a GeoJSON document.

The corresponding client library function is: `getStateGeometry`.

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02`

Returns:

A GeoJSON document.

GET `/state/{fipsCode}/centroids`

Description:

This endpoint will return the centroids for a specified state, identified by its FIPS code.

The corresponding client library function is: `getCountyGeoUnitCentroids`.

NOTE: This endpoint is only well defined, if we are asking for the centroids of a *detailed state*, if a query is directed towards a non-detail state the behavior is not defined.

Parameters:

`{fipsCode}` - This is a **path parameter** which is a string representing the FIPS code of a state. The FIPS code must be padded to 2 digits, IE: `2` => `02`

Returns:

A list of GeoUnitCentroidModel.

Voter Equipment

These endpoints deal with voter equipment related usecases, and be used to access things such as voting equipment models / types, and things such as voting equipment quality.

GET `/votingequipment/`

Description:

This endpoint will return all the voting equipment we have data on, it will always succeed.

The corresponding client library function is: `getAllVotingEquipment`.

Parameters:

N/A

Returns:

A list of VoterEquipmentModel.

GET `/votingequipment/by-manufacturer/{manufacturer}`

Description:

This endpoint will return all the voting equipment we have data on filtering on the specified manufacturer, it will always succeed.

The corresponding client library function is: `getAllVotingEquipmentByManufacturer`.

Parameters:

`{manufacturer}` - This is a **path parameter**, which must be a string representing the manufacturer name.

Returns:

A list of VoterEquipmentModel, which are guaranteed to have the same manufacturer.

GET `/votingequipment/by-type/{type}`

Description:

This endpoint will return all the voting equipment we have data on filtering on the specified machine type, it will always succeed.

The corresponding client library function is: `getAllVotingEquipmentByType`.

Parameters:

`{type}` - This is a **path parameter**, which must be a string representing the equipment type, valid values are:

- `"Batch-Fed Optical Scanner"`
- `"Hand-Fed Optical Scanner"`
- `"Hybrid Optical Scanner/DRE"`
- `"DRE Dial"`
- `"DRE Touchscreen"`
- `"DRE Push Button"`
- `"Remote Ballot Marking"`
- `"Internet Voting"`
- `"Electronic Poll Book"`
- `"BMD"`
- `"BMD/Tabulator"`

Returns:

A list of VoterEquipmentModel, which are guaranteed to be the same equipment type.

GET `/votingequipment/{manufacturer}/{model}`

Description:

This endpoint will return a specific voting equipment by manufacturer and make name.

The corresponding client library function is: `getVotingEquipment`.

Parameters:

`{manufacturer}` - This is a **path parameter** representing the manufacturer of the equipment.

`{model}` - This is a **path parameter** representing the model of the equipment.

Returns:

A single VoterEquipmentModel if it exists, `null` if not.

Detailed Voter Registration

GET `/voter-registration/?stateFips&countyFips`

Description

This will return all the detailed voter registration we have, optionally allowing you to filter by the state & county location to narrow down voter data.

The corresponding client library function is: `getDetailedVoterRegistrationData`

Parameters

`stateFips` - This is a **query parameter**, representing the state fips id. It is `empty` by default.

`countyFips` - This is a **query parameter**, representing the county fips id. It is `empty` by default. If it is provided, then `stateFips` **cannot be null**, otherwise you will get no return value.

Returns A list of VoterRegistrationDataModel.